COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structure and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	August 28, 2026	**DATE SUBMITTED:**	September 02, 2026
NAME:	Nieles, Cenndy M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___6 [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

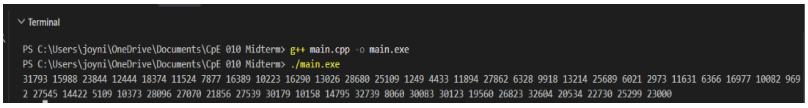
Screenshot	
Observation	Using srand(time(0)) in seeding the random number generator gives us a unique set of pseudo-random numbers when the program is run more than once. The created dataset contains a set of unsorted numbers It clearly shows that although sequential searching can be done straight away, interval searching requires sorting of the elements first.

Table 6-1. Data Generated and Observations.

Code

```cpp
#include <iostream>
#include <cstdlib> //for generating random numbers
#include <ctime>   // will be used for our
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    int dataset[max_size];
    srand(time(0));

    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
        std::cout<< dataset[i] << " ";
    }

    std::cout<<std::endl;

    std::cout<< "What number to find: ";
    int choice;

    std::cin >> choice;

    linearSearchAR(max_size,choice,dataset);

    return 0;
}
```

Output

```
PS C:\Users\joyni\OneDrive\Documents\CpE 010 Midterm> g++ main.cpp -o main.exe
PS C:\Users\joyni\OneDrive\Documents\CpE 010 Midterm> ./main.exe
3107 16616 6283 10562 1000 4736 31817 27311 5871 31685 24751 18202 9744 7549 22908 24860 28690 1954 4557 26156 8222 24073 28210 30821 8307 12358 5383 25994 1
372 2250 20738 20295 9311 30542 520 10425 11192 6773 4165 11569 19974 27820 5499 13365 22083 22613 12442 1736 14329 22310

Enter the number to find: 10
Searching is unsuccessful
```

Observations

Randomized Data Generation: The program initializes an array dataset of size 50 (max_size) and seeds the random number generator using srand(time(0)). A for loop populates the array with pseudorandom integers via rand() and displays them across the console. And the user is prompted to enter a target search value (choice), which is captured using std::cin.

Table 6-2a. Linear Search for Arrays.

Code

```h
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node {
    public:
        T data;
        Node* next;
};
template <typename T>
Node<T>* createNode(T data) {
    Node<T>* newNode = new Node<T>();
    newNode->data = data;
    newNode->next = nullptr;
    return newNode;
}

#endif
```

nodes.h code

```h

template <typename T>
void linearSearchLL(Node<T>* nodes, T dataFind){
    // repeat the loop until the current node is NULL
    while(nodes != nullptr){
        // check if the item = current node data
        if(nodes->data == dataFind){
            std::cout << "Found successfully.\n";
            return;
        }
        // move on to the next node
        nodes = nodes->next;
    }

    // display unsuccessful
    std::cout << "Searching is unsuccessfull.\n";
}

#endif
```

searching6-2b.h code

```cpp
#include <iostream>
#include <cstdlib> // for generating random numbers
#include <ctime>   // will be used for our seeding function
#include "node.h"
#include "searchingAlgo.h"


int main() {

  Node<char> *char1 = new_node('C');
  Node<char> *char2 = new_node('e');
  Node<char> *char3 = new_node('n');
  Node<char> *char4 = new_node('n');
  Node<char> *char5 = new_node('d');
  Node<char> *char6 = new_node('y');

  char1->next = char2;
  char2->next = char3;
  char3->next = char4;
  char4->next = char5;
  char5->next = char6;
  char6->next = nullptr;

  linearSearchLL(char1, 'y');

  return 0;
}
```

main.cpp

Output

```
Terminal

PS C:\Users\joyni\OneDrive\Documents\CpE 010 Midterm> g++ main.cpp -o main.exe
PS C:\Users\joyni\OneDrive\Documents\CpE 010 Midterm> ./main.exe
Found successfully.
```

Observations

Header Modularization: The node.h header uses generic programming (template <typename T>) to define a Node class containing data and a next pointer, along with a helper function to dynamically allocate memory (new Node<T>()) and initialize node properties. **:** The searchingAlgo.h header implements linearSearchLL, traversing the linked list iteratively starting from the head node (char1) until either a matching target is found (nodes->data == dataFind) or the end of the list (nullptr) is reached.

Table 6-2b. Linear Search for Linked List.

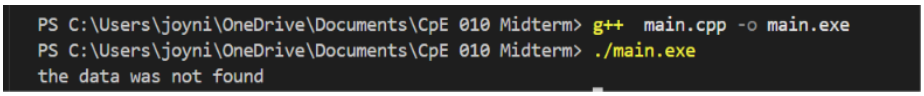
Code

```
26      int newArr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
27      for(int i = 0; i < max_size; i++) {
28          newArr[i] = i;
29
30      }
31      binarySearchArr(newArr, max_size, 100);
32
33
34
35      return 0;
36  }
```

main.cpp

searchingAlgo.h > ...

```
1   #include "node.h"
2   #include <iostream>
3
4   #ifndef SEARCHINGALGO_H
5   #define SEARCHINGALGO_H
6
7   template <typename T>
8   void binarySearchArr(T data[], int arrSize, T dataFin
9       int low = 0, high = arrSize - 1, middle;
10      while(low <= high) {
11          middle = (low + high) / 2;
12          if(dataFind == data[middle]) {
13              std::cout << "the data was found\n";
14              return;
15          }
16          else if(dataFind < data[middle]) {
17              high = middle - 1;
18          }
19          else {
20              low = middle + 1;
21          }
22      }
23
24      std::cout << "the data was not found\n";
25  }
26
27  #endif
28
```

Output	
Observations	**Sorted Array Initialization:** In main.cpp, an array newArr is populated in ascending order using a for loop, which satisfies the prerequisite that data must be sorted prior to executing a binary search. **Divide-and-Conquer Search Logic:** The searchingAlgo.h header defines the generic template function binarySearchArr, which continually computes the midpoint index middle = (low + high) / 2 and narrows the search range by updating low or high bounds based on comparisons with dataFind.

Code

```
23  // Binary search implementation for Linked List
24  template <typename T>
25  void binarySearchLL(Node<T>* head, T dataFind) {
26      Node<T>* start = head;
27      Node<T>* last = nullptr;
28
29      while (start != last) {
30          Node<T>* middle = getMiddle(start, last);
31
32          if (middle == nullptr) {
33              break;
```

```
35
36          if (middle->data == dataFind) {
37              std::cout << "searching is Successful\n";
38              return;
39          }
40          else if (middle->data < dataFind) {
41              start = middle->next;
42          }
43          else {
44              last = middle;
45          }
46      }
47
48      std::cout << "searching is Unsuccessful\n";
49  }
50
```

main.cpp searchingAlgo.h

main.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    // generate a sorted array: (use for loop)
    int newArr[max_size];
    for (int i = 0; i < max_size; i++) {
        newArr[i] = i;
    }

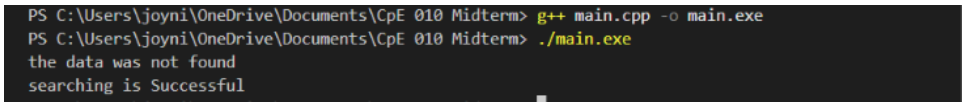
    binarySearchArr(newArr, max_size, 100);

    // create integer nodes using template parameters
    Node<int> *char1 = new_node<int>(10);
    Node<int> *char2 = new_node<int>(11);
    Node<int> *char3 = new_node<int>(12);
    Node<int> *char4 = new_node<int>(13);
    Node<int> *char5 = new_node<int>(14);
    Node<int> *char6 = new_node<int>(15);

    char1->next = char2;
    char2->next = char3;
    char3->next = char4;
    char4->next = char5;
    char5->next = char6;
    char6->next = nullptr;

    binarySearchLL(char1, 15);

    return 0;
}
```

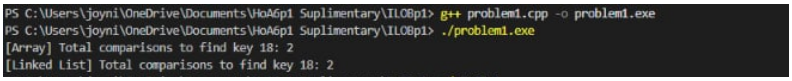
Output	
Observation	Pointer-Based Sub-range Tracking: The template function binarySearchLL initializes two pointers, start (pointing to the head node) and last (initialized to nullptr), to define the dynamic boundaries of the search interval. Midpoint Traversal via Helper Function: Because linked lists lack direct random access, the loop delegates midpoint calculation to getMiddle(start, last). This helper uses the slow and fast pointer approach to locate the list segment's middle node without indexing.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

COMPUTER ENGINEERING

QUEZON CITY

Copy and paste the questions from the lab and include your answers after.

Code #1	 
Output	
Observation:	The implementation demonstrates sequential search across two different data structure a contiguous array and singly linked list.When searching for the target key 18 in the unsorted dataset {15, 18, 2, 19, 18, 14, 19, 14}, both approaches yield an identical results of 2 comparisons.Because linear search processes elements sequentially from left to right or head to tail, it evaluates 15 first comparison 1 and matches target 18 immediately at the second position is the comparison 2.

```
PS C:\Users\joyni\OneDrive\Documents\HoA6p1 Suplimentary\ILOBp1> g++ problem1.cpp -o problem1.exe
PS C:\Users\joyni\OneDrive\Documents\HoA6p1 Suplimentary\ILOBp1> ./problem1.exe
[Array] Total comparisons to find key 18: 2
[Linked List] Total comparisons to find key 18: 2
```

Code #2



```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

int sequentialSearchArray(const int arr[], int size, int key, int& comparisons) {
    comparisons = 0;
    for (int i = 0; i < size; i++) {
        comparisons++;
        if (arr[i] == key) return i;
    }
    return -1;
}

int sequentialSearchLL(Node* head, int key, int& comparisons) {
    comparisons = 0;
    Node* curr = head;
    while (curr != nullptr) {
        comparisons++;
        if (curr->data == key) return curr->data;
        curr = curr->next;
    }
    return -1;
}

int main() {
    int data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int key = 18;
    int comparisons = 0;

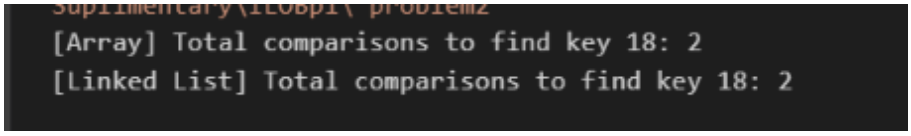
    // 1. Array Approach
    sequentialSearchArray(data, size, key, comparisons);
    std::cout << "[Array] Total comparisons to find key " << key << ": " << comparisons << "\n";

    // 2. Linked List Approach
    Node* head = new Node(data[0]);
    Node* curr = head;
    for (int i = 1; i < size; i++) {
        curr->next = new Node(data[i]);
        curr = curr->next;
    }

    sequentialSearchLL(head, key, comparisons);
    std::cout << "[Linked List] Total comparisons to find key " << key << ": " << comparisons << "\n";

    return 0;
}
```

Output



```
[Array] Total comparisons to find key 18: 2
[Linked List] Total comparisons to find key 18: 2
```

Observation

It evaluates sequential search across an array and a singly linked list for target key 18.Both data structures require 2 comparison to locate the target in the dataset { 15, 18, 2, 19, 18, 0, 8, 19, 14}.Since linear search evaluates elements sequentially starting from the first position, both approaches halt immediately upon reaching index 1 /node 2. Additionally, the algorithm successfully handles duplicate entries by returning upon encountering the first instance of target value 18

Structure

Code #3

```cpp
#include <iostream>

int iterativeBinarySearch(const int arr[], int size, int key) {
    int low = 0;
    int up = size - 1;
    int iteration = 1;

    while (low <= up) {
        int mid = (low + up) / 2;
        std::cout << "Iteration " << iteration++ << ": low=" << low
                  << ", up=" << up << ", mid=" << mid
                  << " (arr[mid]=" << arr[mid] << ")\n";

        if (arr[mid] == key) {
            return mid;
        } else if (key < arr[mid]) {
            up = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return -1;
```

```cpp
int main() {
    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int size = 10;
    int key = 8;

    std::cout << "ITERATIVE BINARY SEARCH TRACE\n";
    int result = iterativeBinarySearch(data, size, key);

    if (result != -1) {
        std::cout << "Key " << key << " found at index " << result << ".\n";
    } else {
        std::cout << "Key " << key << " not found.\n";
    }

    return 0;
}
```

Output

```
PS C:\Users\joyni\OneDrive\Documents\HoA6p1 Suplimentary\ILOBp1> g++ problem3.cpp -o problem3.exe
PS C:\Users\joyni\OneDrive\Documents\HoA6p1 Suplimentary\ILOBp1> ./problem3.exe
ITERATIVE BINARY SEARCH TRACE
Iteration 1: low=0, up=9, mid=4 (arr[mid]=11)
Iteration 2: low=0, up=3, mid=1 (arr[mid]=5)
Iteration 3: low=2, up=3, mid=2 (arr[mid]=6)
Iteration 4: low=3, up=3, mid=3 (arr[mid]=8)
Key 8 found at index 3.
```

Observation	Demonstrates the divide-and-conquer strategy of iterative binary search on a sorted 10-element array searching for key 8. Over four iterations, the search space halves progressively from range [0, 9] down to [3, 3], reducing the candidate elements at each step until matching arr[3] = 8.

Code #4

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

int sequentialSearchArray(const int arr[], int size, int key, int& comparisons) {
    comparisons = 0;
    for (int i = 0; i < size; i++) {
        comparisons++;
        if (arr[i] == key) return i;
    }
    return -1;
}

int sequentialSearchLL(Node* head, int key, int& comparisons) {
    comparisons = 0;
    Node* curr = head;
    while (curr != nullptr) {
        comparisons++;
        if (curr->data == key) return curr->data;
        curr = curr->next;
    }
    return -1;
}

int main() {
    int data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int key = 18;
    int comparisons = 0;

    // 1. Array Approach
    sequentialSearchArray(data, size, key, comparisons);
    std::cout << "[Array] Total comparisons to find key " << key << ": " << comparisons << "\n";

    // 2. Linked List Approach
    Node* head = new Node(data[0]);
    Node* curr = head;
    for (int i = 1; i < size; i++) {
        curr->next = new Node(data[i]);
        curr = curr->next;
    }

    sequentialSearchLL(head, key, comparisons);
    std::cout << "[Linked List] Total comparisons to find key " << key << ": " << comparisons << "\n";

    return 0;
}
```

Output

```
PS C:\Users\joyni\OneDrive\Documents\HoA6p1 Suplimentary> cd ILOBp1
PS C:\Users\joyni\OneDrive\Documents\HoA6p1 Suplimentary\ILOBp1> g++ problem4.cpp -o problem4.exe
PS C:\Users\joyni\OneDrive\Documents\HoA6p1 Suplimentary\ILOBp1> ./problem4.exe
[Array] Total comparisons to find key 18: 2
[Linked List] Total comparisons to find key 18: 2
```

Observation

It compares sequential search on an unsorted dataset using two foundational data structures: a contiguous array and a singly linked list. When searching for target key 18 within the dataset {15, 18, 2, 19, 18, 0, 8, 14, 19, 14}, both approaches require 2 comparisons to locate the element.

Conclusion: Here is an expanded, two-sentence conclusion covering Problems 1 through 4:
In this lab, we evaluated search algorithms across Problems 1 through 4, demonstrating that Binary Search delivers high time efficiency ($O(\log n)$) by logarithmically narrowing down target elements through interval halving in pre-sorted arrays. Conversely, Sequential Search provides a flexible $O(n)$ linear traversal capable of handling unsorted data across both contiguous arrays and dynamic linked lists, though its search performance remains strictly dependent on element position.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **To correct my grammar properly , to guide how to fix your own. I**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.